IOT Based Trouble Ticketing System

By

Rohan Rajeshbhai Gohel

24MCA020

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
Institute of Technology, Nirma University
Ahmedabad – 382481

IOT Based Trouble Ticketing System

Internship Report

Submitted in partial fulfillment of the requirements
For the degree of

Master of Computer Application

By

Rohan Rajeshbhai Gohel

24MCA020

Guided by

Dr. Saurin parikh

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
Institute of Technology, Nirma University
Ahmedabad – 382481

CERTIFICATE

This is to certify that the Internship project entitled “IOT Based Trouble Ticketing System” submitted by Gohel Rohan Rajeshbhai(24MCA020), towards the partial fulfillment of the requirements for the degree of Master of Computer Application of Nirma University is the record of work carried out by him under my supervision and guidance. In my opinion, the submitted work has reached a level required for being accepted for examination.

Dr. Saurin Parikh
Associate Professor
Computer Science and Eng. Dept.,
Institute of Technology,
Nirma University,
Ahmedabad

Dr Ankit Thakkar
Professor and I/C HOD,
Computer Science and Eng. Dept.
Institute of Technology,
Nirma University,
Ahmedabad

Statement of Originality

I, Gohel Rohan Rajeshbhai, Roll no: 24MCA020, give undertaking that the Internship Project entitled "**IOT Based Trouble Ticketing System**" submitted by me, towards the partial fulfilment of the requirements for the degree of Master of Computer Application of Institute of Technology, Nirma University, Ahmedabad, contains no material that has been awarded for any degree or diploma in any university or school in any territory to the best of my knowledge. It is the original work carried out by me and I give assurance that no attempt of plagiarism has been made. It contains no material that is previously published or written, except where reference has been made. I understand that in the event of any similarity found subsequently with any published work or any dissertation work elsewhere; it will result in severe disciplinary action.

Signature of Student

Date:

Place:

Endorsed by

Dr.SaurinParikh
(Signature of Guide)

Acknowledgement

First of all, it is important for me to thank everyone who has been with me and assisted me through the entire process of developing the IOT Based Trouble Ticketing System Project.

It is important for me to convey my heartfelt gratitude to my internal guide, Prof .SaurinParikh, for encouraging me throughout this process. His valuable feedback and constant support have made me learn many things from this process and have helped me develop better problem-solving skills.

I must thank all the faculty members of the department of Computer Science and Engineering, Institute of Technology, Nirma University for creating such an environment that allowed for learning. Throughout the course of MCA, their assistance had always been instrumental in undertaking a project of this caliber.

Finally, I would like to thank my friends who have constantly helped me out during difficult stages of this project by giving me suggestions and encouraging me throughout. Moreover, the open source developers whose code and libraries I have used for building this application. They have made it very convenient for me to implement something on such a level. Their efforts are commendable indeed.

ABSTRACT

Objective of Work

The IoT-Based Trouble Ticketing System can be considered another step towards automation in facility management and asset tracking since it is meant to assist in the automated report generation of problems. This project was developed as an internship assignment within 16 weeks at Nirma University where the use of the ESP32 microcontroller as the basis for hardware-to-software integration was employed.

As part of the project, the main focus was put on the development of a system that will replace traditional manual log entry with an automatic process referred to as "barcode-to-ticket". In the current version, this process is accompanied by a user-friendly menu system with the LCD display where all necessary settings have already been made and are now frozen, ensuring no one can tamper with them accidentally.

From a hardware perspective, the developed system includes a camera module for barcodes and an LDR sensor for environmental detection. As for environment testing, it was performed extensively to ensure scanning reliability under different light intensity levels; as for system hardening, measures have been taken to harden the housing of the ESP32 to prevent vibration. On the software side, the state machine approach was chosen to transition between two states: from "waiting" to "sent"..

CONTENT

Chapter 1 Introduction

1.1 About the Organisation

1.1.1 Introduction of the Institution

The present project has been conducted as an internal academic activity at Nirma University,Ahmedabad, which came into existence in the year 2003. It is a prominent educationalinstitution imparting quality education in various fields through undergraduate, postgraduate,and doctorate courses. Nirma Institute of Technology offers Master of ComputerApplications (MCA) programme where emphasis has been laid upon the development oftechnical knowledge and practical exposure of the students.
At the institution, there is constant promotion of innovation among students in order to enable them to perform real projects that increase their ability to solve practical problems. Therefore, the academic environment of Nirma University has proved to be highly suitable for such a project as Trouble ticketing system (IOT). The students have enough infrastructure and academic freedom at their disposal for designing and building systems of practical significance.

1.1.2 Quality Policy

Nirma University aims to maintain high standards of education and constant improvement in

the academic process. Outcome-Based Education (OBE), continuous internal evaluation, and academic reviews are undertaken by the institution to ensure continuous attainment of academic objectives. The institute has been accredited by the University Grants Commission (UGC) and stresses quality education along with its industrial relevance. requirements, ensuring both theoretical understanding and practical application in every discipline it offers.

1.1.3 Communication

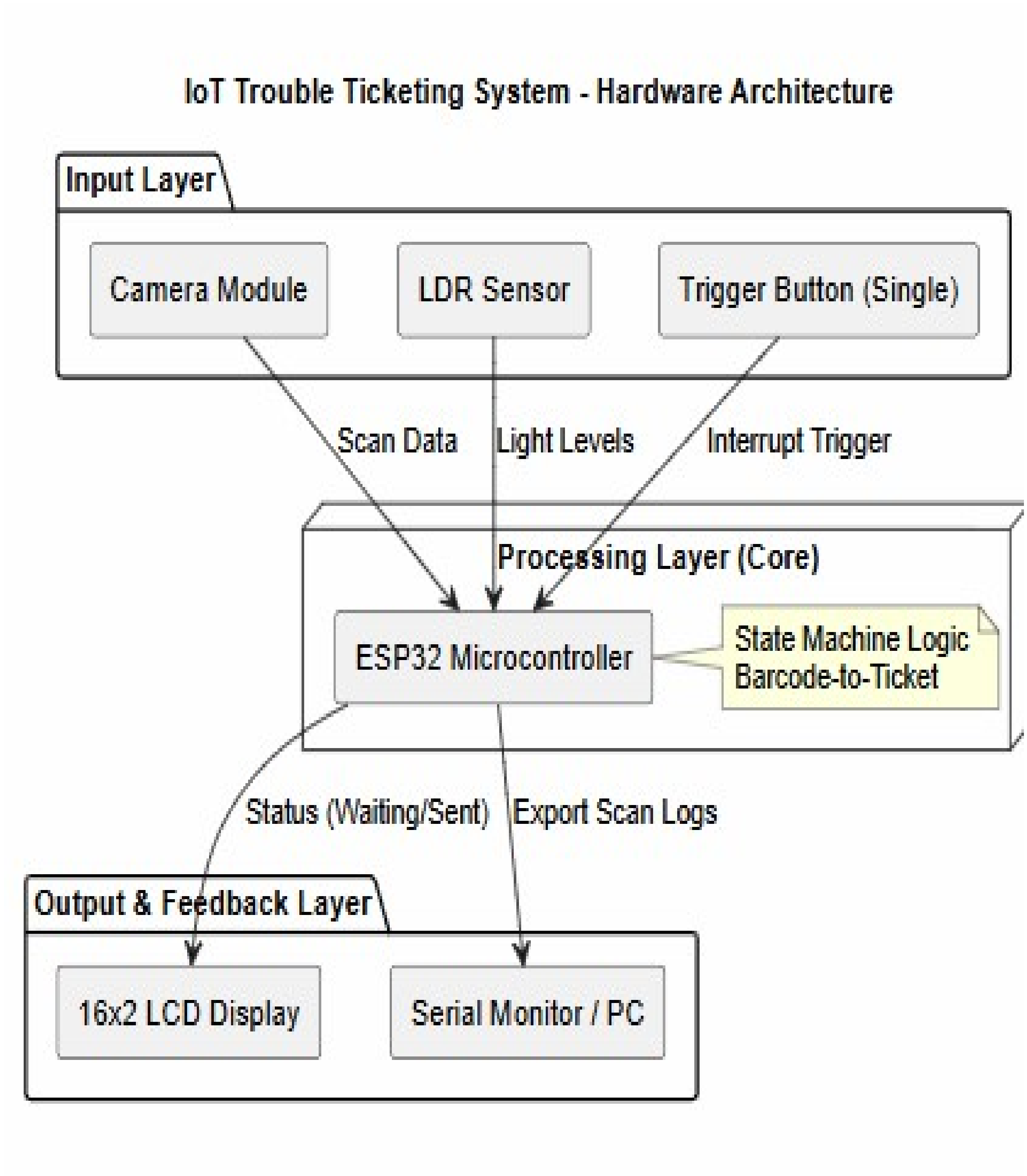
Regular and effective communication between the student and the guide regarding the project
was maintained. Periodic meetings and review sessions helped ensure timely and efficient progress
of the work along with addressing any issues and giving structured feedback at each stage of the-1
development process. Progress reports were made by email and presentations and helped coordinate
effectively between the two, thus allowing the successful completion of all milestones of the
project.

1.1.4 Resources

For the implementation of this project, institutional resources such as computer facilities, high-speed
internet connections, and software and other development tools needed for implementation were
used. The Department of Computer Science and Engineering supplied all technical references and
laboratory facilities needed in the course of the project. Assistance and supervision of faculty members
were invaluable in the course of carrying out this project successfully.

1.2 The System

1.2.1 Definition of System



1.2.1 **Diagram of system**

1.2.2 Objectives

The main objective of the IoT-Based Trouble Ticketing System is to bring automation into the facility management system process of Nirma University through an automated, hardware-based trouble reporting system.

Automated issue reporting: For automating the "barcode to ticket" process, where the system can identify the asset automatically and generate the ticket through the barcode.

•Hardware Software Interaction: To provide a seamless way of interaction between the ESP32 microcontroller and other visual input devices such as Camera.

•Efficiency Improvements: To improve the time taken in repairing issues as the system's status will change from Waiting to Sent immediately after verification.

•User Friendly Design: For simplifying the design process through a finalized LCD menu design and minimal button usage for reducing user errors.

1.2.3 Description of Present System

An IoT-Based Trouble Ticketing System is being introduced, which brings automation to the process of reporting problems in the facility management system. This system makes use of automation and hardware software interaction for achieving its objectives

- Status of Current Prototype: At the moment, the system is fully functional, being comprised of hardware and components, such as ESP32 microcontroller and the camera module used for scanning barcodes.

- Fundamental Functionality: The project has already succeeded in implementing the logic "from barcode to ticket," meaning that the system will be able to identify assets and show status messages on the finished 16x2 LCD display, such as "Waiting" or "Sent."

- System Preparation: In addition, we have programmed and tested the firmware of the project, which makes use of libraries to perform various operations, such as sensing light intensity and user interaction with the help of the only button available.

- Security of Assembly: In order to protect the system from any physical damage or vibration when scanning a barcode, we have provided the assembly with a case, which will make sure that ESP32 microcontroller and camera module stay safe

1.2.4 proposed system

The current stage of our project includes working with hardware and its assembling; however, we can expand the functionality of our system greatly:

- PCB Manufacturing: We can switch from the assembly with jumpers to PCB, which will improve the appearance and robustness of the device.

- Cloud Connectivity: Future versions would rely on the Wi-Fi connection capability provided by ESP32 to connect to cloud platforms such as AWS IoT or Google Cloud Services. It would allow logging of scans into the cloud as well as real-time dashboard viewing from anywhere.
- Software Connectivity: The proposed system is also capable of being integrated with an organization's ERP or helpdesk system through API connections, enabling "Sent" Tickets to initiate a Maintenance Order in the company's database.
- Power Circuit Optimizations: More efficient power circuit designs could be added to make the system portable using battery cells, making it a hand-held device for comprehensive facility inspection.

1.3 Project Profile

- **1.3.1 Project Title**
- **IOT Based Trouble Ticketing System**
- **1.3.2 Scope of Project**
- The scope of the proposed IoT-Based Trouble Ticketing system is the development of the hardware prototype of the system to achieve automated reporting of technical issues.
- Hardware: Building an ESP32 board that includes a Camera module and LDR sensor to perform barcode ticketing logic.
- Logical Components: Implementation of state machines from "Waiting" state to "Sent" State following proper scanning of barcodes.
- UI/UX Components: Implementation of a local feedback loop using an LCD monitor and a one-button interface.
- **Test Environment: Environmental testing including scan accuracy and vibration hardening test cases.Data**
- **Logging:** Developing scripts to export scan logs from the ESP32 to a serial monitor for record-keeping.
- **Future Readiness:** Designing the architecture to allow future upgrades to **PCB design, Cloud connectivity**, and external software integration.

1.3.3 project team

Sr.	Name	Role
1	Rohan Gohel	Developer/Intern
2	Dr.saurin parikh	Internal Guide

1.3.4 Hardware/Software Environment

Category	**Component**	**Description**
Microcontroller	**ESP32**	Core processing unit with integrated Wi-Fi for logic execution.
Sensors	**Camera Module**	Used for barcode scanning and asset identification.
Sensors	**LDR Sensor**	Monitors ambient light to ensure scanning accuracy.
Display	**16x2 LCD**	Displays system states such as "Waiting" and "Sent".
Input	**Single Push Button**	Streamlined trigger for starting the scan process.
Prototyping	**Breadboard & Wires**	Used for physical assembly and pinout testing.
IDE	**Arduino IDE**	Primary software environment for coding and uploading firmware.
Language	**C++**	Used for developing the state machine and hardware control.
Library 1	**ESP32 Camera**	Manages frame capture and barcode-to-ticket processing.
Library 2	**LiquidCrystal I2C**	Facilitates status message display with minimal wiring.

Chapter 2 System Analysis

2.1 Feasibility Study

2.1.1 Operational Feasibility

The operational feasibility of the IoT-Based Trouble Ticketing System assesses its effectiveness in addressing the current problem as well as the degree of ease with which it can be implemented in the daily activities.

Ease of Use: The ease of use of this system comes with the simple user interface design and the inclusion of a single button to scan the items.

Real-Time State Notifications: Immediate notification of user activity is provided through the LCD menu with the indication of states, from “Waiting” to “Sent.”

•Automation: The implementation of barcode to ticket logic has enabled automatic reporting, thus eliminating the need for data entry and streamlining the entire reporting process.

•Scalability: This feature enables future expansion to include software and cloud connection.

2.1.2 Technical Feasibility

The technical feasibility of the system shows that the required hardware and software resources exist and are sufficient to achieve the expected results from the proposed project.

- Hardware Capacity: The capabilities of the ESP32 microcontroller are adequate to handle both the "barcode to ticket" mechanism and the parallel input from sensors.

- Data Visualization Handling: Application of the special ESP32 Camera library makes it possible to take an effective picture and validate the barcode within the embedded system.

- System Stability: Implementation of the system optimization measures, such as elimination of unnecessary debug lines and effective memory handling, ensures the stability of the device.

- Interface Efficiency: Changeover from multiple buttons interface to one button was made by means of implementing interrupts, which simplified the design of the hardware part.

Protection from Physical Damage: Technical solutions to ensure physical protection of the ESP32 box guarantee safety of its components from potential shaking.

Development Platform: The Arduino IDE and C++ language facilitated effective development and debugging of the system's firmware.

2.1.3 Financial and Economical Feasibility

•Low Component Cost: Application of cheap ESP32 board and Camera module enabled creating a very efficient system with limited initial financial expenses.

•Liberation from Licensing Costs: By opting for the free software and libraries, including LiquidCrystal I2C, the system avoids licensing costs.

•Operational Economy: Automated solution saves time and effort necessary for manual registration of ticket sales and maintenance of the register.

Reduced Costs of Components: Moving to one-button operation lowered the costs associated with physical components and made the system wiring simpler to facilitate manufacturing.

Longevity: Hardening the system and protecting the ESP32 case will increase the longevity of the

electronics by ensuring they can withstand vibrations, thus lowering replacement frequencies.

2.2Requirement Analysis

2.2.1Facts-Finding Techniques

Interview

The informal interviews conducted targeted possible users of the system, including students and faculty members at Nirma University. These individuals handle facility management and technical support. The conversations revolved around their experience in reporting any technical problems. One of the key challenges mentioned by interviewees is that it takes too much time to log a technical problem on paper. Therefore, they might report an issue late, and there is no way to track its progress in real-time. Another major challenge they face when logging a problem is identifying the serial number of the faulty equipment incorrectly Moreover, there was a significant number of users who observed that the inability to see clearly and vibrations in the surroundings hindered their ability to enter asset data correctly. Users were keenly interested in having an automatic system that could create a completely validated trouble ticket just by scanning the bar code with little to no need for manual effort. Other users also mentioned that the presence of a visual indication like the system changing its state from "Waiting" to "Sent" in a specific LCD screen will make the system more credible

Questionnaire

A short questionnaire was sent out to about 20 respondents from Nirma University as a means of complementing the information obtained from the interview. The questions comes around how often users experience technical problems, how they currently submit data, and which attributes would be most valuable in an automated problem-solving theory. Itinerary logging and environmental accuracy were identified as the two attributes of most concern, while a simple user interface that can still function even with physical vibrations came in third place compensation

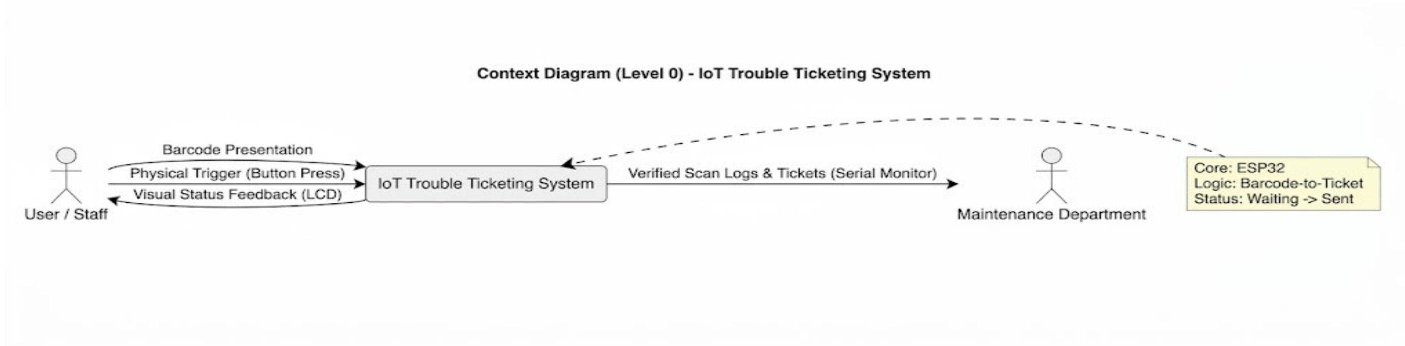
Record review

The record review included reviewing past Daily Work Summaries (DWS) and Internship Activity Reports (IAR) in order to get acquainted with the technical difficulties that occurred repeatedly and milestones that proved to be successful. The review involved examining the historical progress of the 'barcode-to-ticket' logic with particular emphasis on the transition from the first environment testing to the final creation of the state machine. Through the analysis of previous scan logs and documentation review, it was noted that there were some major obstacles regarding the lighting conditions and stability of the system. Thanks to historical records, it was decided to use an LDR sensor in order to compensate for lighting issues, as well as harden the system so that the ESP32 unit would be protected against vibrations. Last feedback records provided during work at Nirma University were also useful in improving the user experience, resulting in the locking of the LCD menu for preventing unwanted changes in its configuration Observed.

Direct watch over to see how users interact with hardware device in the practical environment. It was noticed that one of the biggest problems was the difficulty associated with, leading to unaexpected resets or incorrect feature navigation. This physical observation of user frustration directly resulted in the technical decision to streamline the interface into a **single-button** trigger system

2.3 Context diagram

-



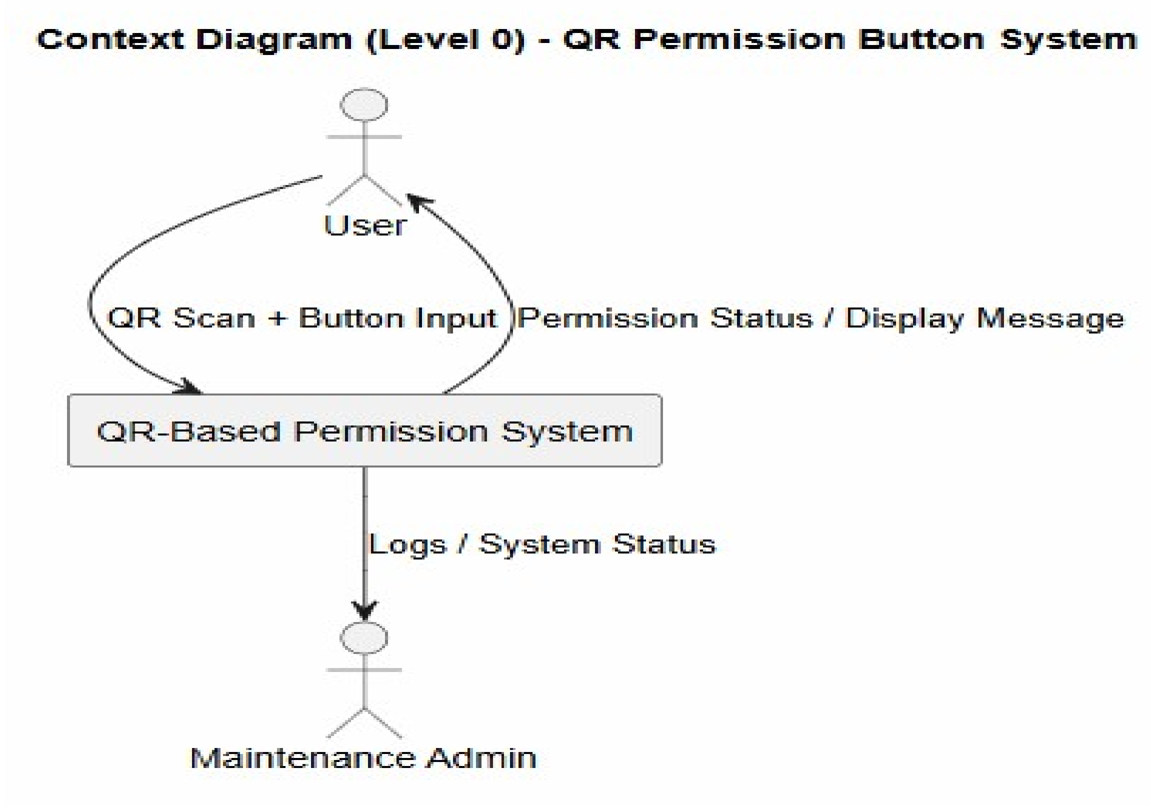
2.3Context diagram

Description for your Report:

Context Diagram:This diagram provides the topmost level of the IoT-Based Trouble Ticketing System which represents that system as one process communicating with other entities as:

•User: Initiates process by clicking on the single-button interface and gets the output on real-time basis from the LCD menu screen.• System: Serves as the main hub and performs the environmental correctness checks and the barcode to ticket mapping logic through Camera Module.• Maintenance Department (Admin): Gets the final output from the validated scanned logs generated by the ESP32 and available on a serial monitor or main database

2.4 Data Flow Daigram

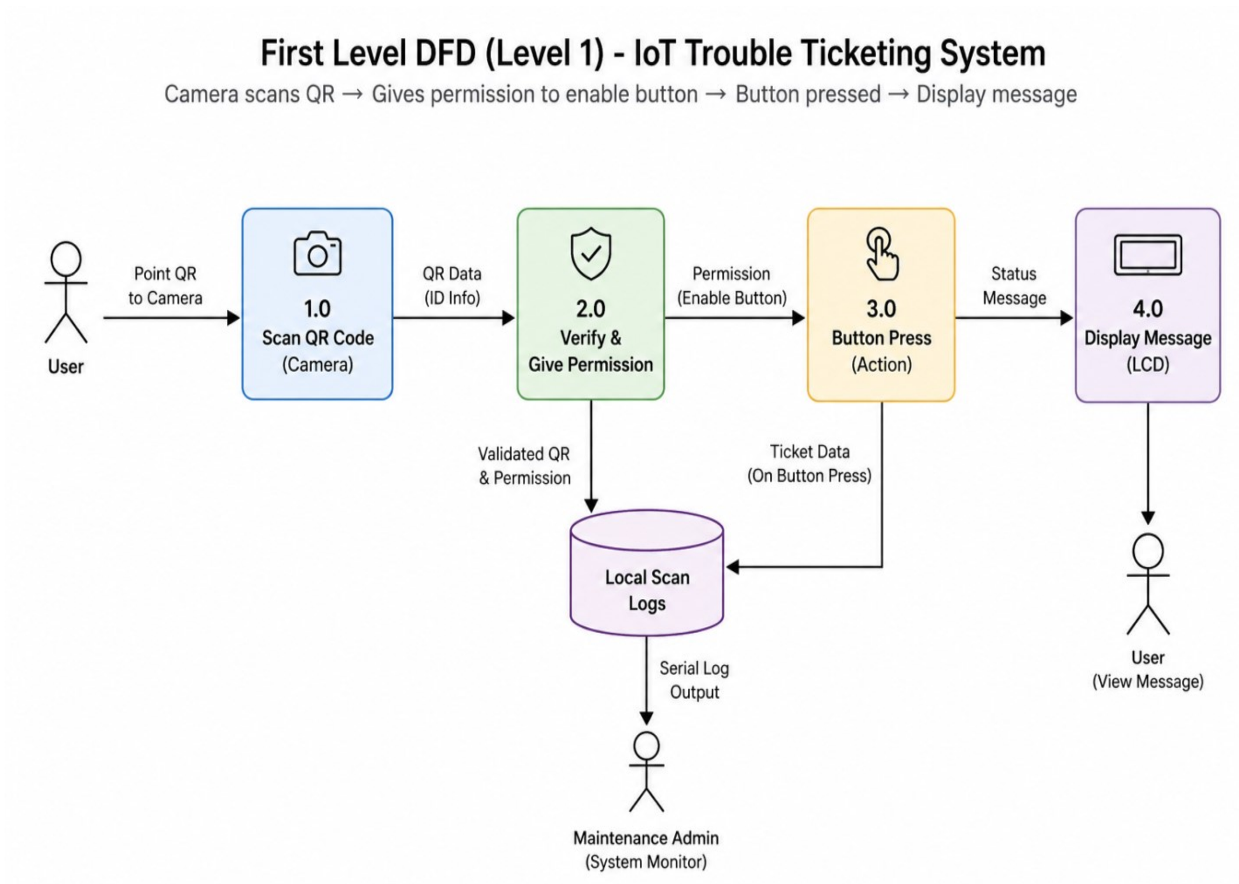


2.4.1 0-LevelDFD

Level 0 Data Flow Diagram (DFD):

The level 0 DFD includes all the components involved in the QR-based permission system working as a single process. According to this diagram, a user will use a camera module in order to scan a QR code and hit the button. Then the system will process the requests made by the user and provide a result in the form of permissions and log details to the maintenance department

2.4.2 First Level DFD



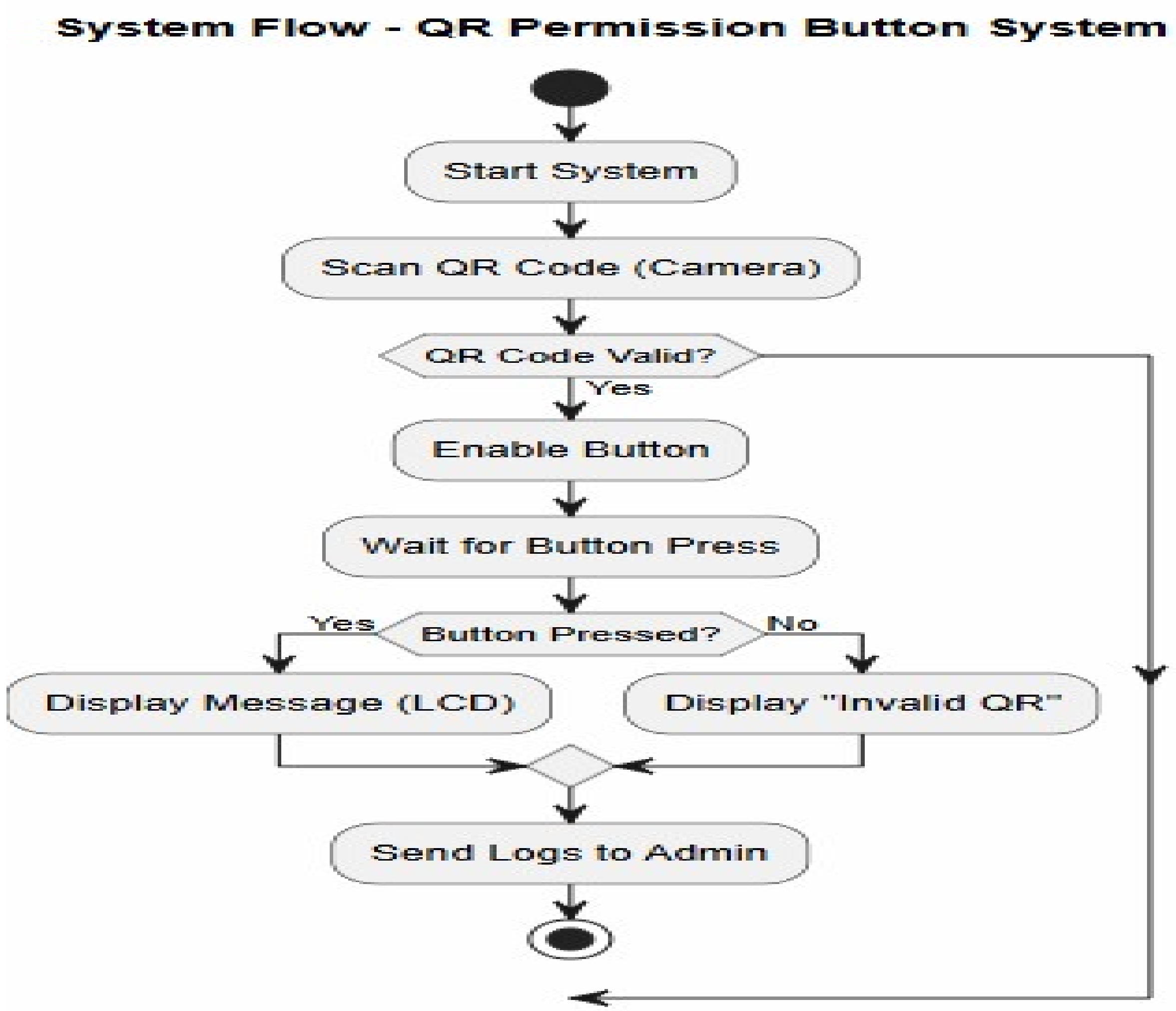
2.4.2 **First Level DFD**

Level 1 DFD for QR Permission Button System:

In Level 1 DFD, we can understand the stepwise working of the QR-based permission button system. Firstly,, The system checks the validity of the QR code, and upon verification, the system allows the activation of the button. After activating the button, the user clicks the button, prompting the system to take a certain action. The last process involves displaying a message to the user through a display screen. All the processes are stored locally for the admin to monitor.

Chapter 3 System Design

3.1 System Flow

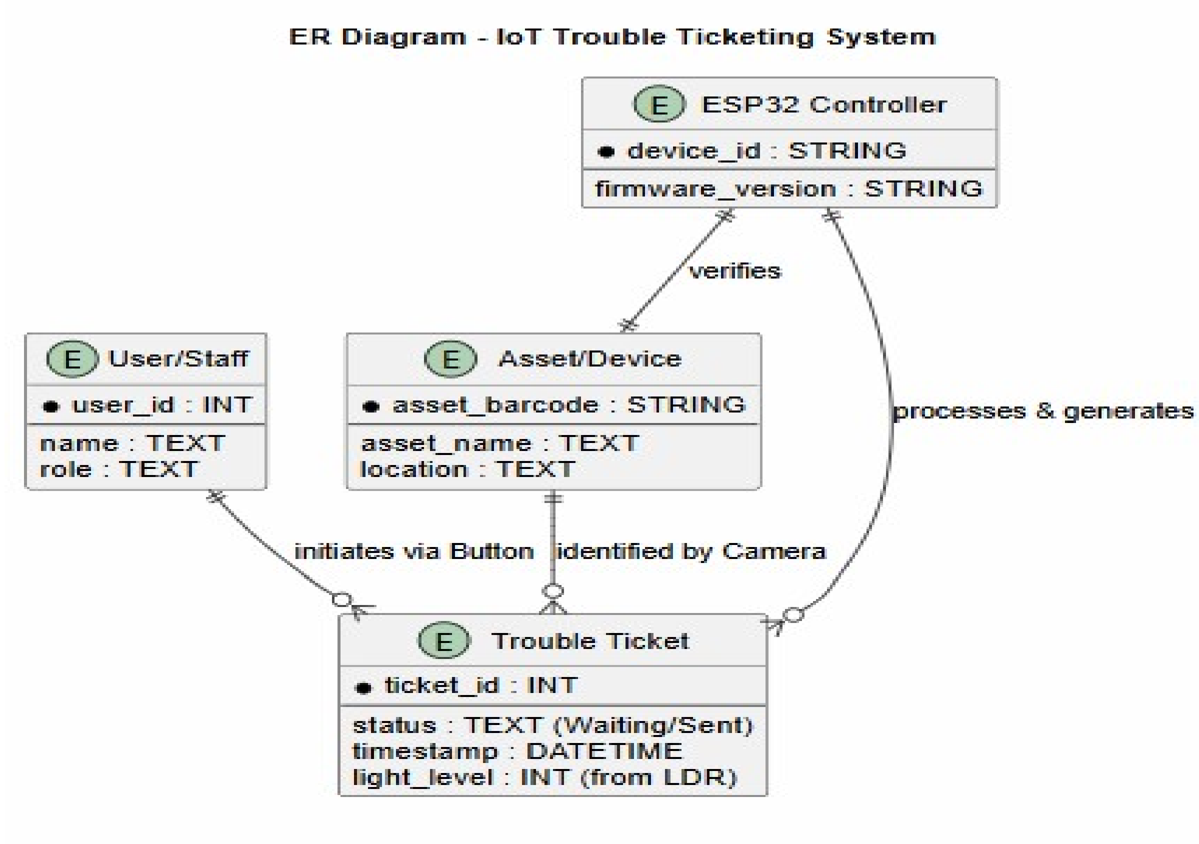


3.1 Systeflow

System flow diagram of QR-based permission button system

A system flow diagram of the QR-based permission button system shows the working steps of the system. First, the system uses a camera to scan a QR code. Then it verifies whether the QR code is correct or incorrect. If it is correct, the system activates the button and waits for it to be pressed. Once the button is pressed, it will show a message on the screen and save the information in the logs. However, if it is incorrect, the system immediately shows an "Invalid QR" message. All data collected and saved in the logs are transmitted to the administrator for analysis.

3.2 Entity-Relationship Diagram



3.2ER Diagram

Description of entities:

User: The person start the system using the one-button press.

Device: Physical idcard capture by the camera module using its unique barcode value.

Trouble Ticket: Digital trouble ticket generated from the successful scan with "Sent" status and ambient light level from the LDR sensor.

ESP32 Controller: Centralized hardware device responsible for the algorithmic logic and transitioning between the "Waiting" and "Sent" states.

Data Dictionary

Data Name	Description	Type	Source	Destination
QR_Code_Data	Encoded data captured from QR scan	String	User (Camera)	Verification Process
QR_Status	Result of QR validation (Valid/Invalid)	Boolean/String	Verification Process	System Flow
Permission_Status	Indicates whether button is enabled or not	Boolean	System	Button Module
Button_Input	Signal when user presses the button	Boolean	User	System
Display_Message	Message shown on LCD (Success / Error)	String	System	User (Display)

Chapter 4 Results and Discussion

4.1 Results

This chapter presents the outcomes of the **IoT-Based Trouble Ticketing System** implementation. Each major functional component is described through its observed behavior, demonstrating the system's ability to transition from a physical scan to a digital record.

4.2 library setup

LiquidCrystal I2C: Look for the one by Frank de Brabander. This controls the green screen in your photo.

- **ESP32QRCodeReader:** This is the most stable library for using the camera to "read" data without needing an external computer.
- **ArduCAM:** If it is an ArduCAM (SPI pins like CS, MOSI), you need the ArduCAM library.

4.3 Arduino IDE Setup (ESP32 Board Package)

1. Go to **Tools > Board > ESP32.**
2. Select **ESP32 Dev Module.**
3. If you don't see "ESP32" in the menu:
   - Go to **File > Preferences.**
   - In "Additional Boards Manager URLs", paste: https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
   - Go to **Tools > Board > Boards Manager**, search for "ESP32", and click **Install.**

Chapter 5 IOT TTS core overview

5.1 Port details

LCD to ESP32 / Power Wiring Map

- **Pin 1 (VSS):** Connect to **GND** (Blue rail).
- **Pin 2 (VDD):** Connect to **5V** (from VIN or USB / Red rail).
- **Pin 3 (VO):** Connect to **GND** or a Potentiometer to adjust contrast.
- **Pin 4 (RS):** Connect to **P15 (GPIO 15)** on the ESP32.
- **Pin 5 (RW):** Connect to **GND.**
- **Pin 6 (E):** Connect to **P2 (GPIO 2)** on the ESP32.
- **Pin 7 - 10 (D0-D3): Leave Empty** (not used in 4-bit mode).
- **Pin 11 (D4):** Connect to **P4 (GPIO 4)** on the ESP32.
- **Pin 12 (D5):** Connect to **P16 (RX2 label)** on the ESP32.
- **Pin 13 (D6):** Connect to **P17 (TX2 label)** on the ESP32 (Standard 4-bit sequence).
- **Pin 14 (D7):** Connect to **P5 (D5 label)** on the ESP32 (Standard 4-bit sequence).
- **Pin 15 (Backlight +):** Connect to **5V** (Red rail).
- **Pin 16 (Backlight -):** Connect to **GND** (Blue rail).

Device Pin	ESP32 Board Label	Arduino Code Pin
Camera	CS P5 (or D5)	5
Camera	MOSI P23(or D23)	23

Camera	MISO P19 (or D19)	19
Camera	SCK P18 (or D18)	18
LCD/Cam	SDA P21 (or D21)	21
LCD/Cam	SCL P22 (or D22)	22
Button	P13	13

5.2 code overview

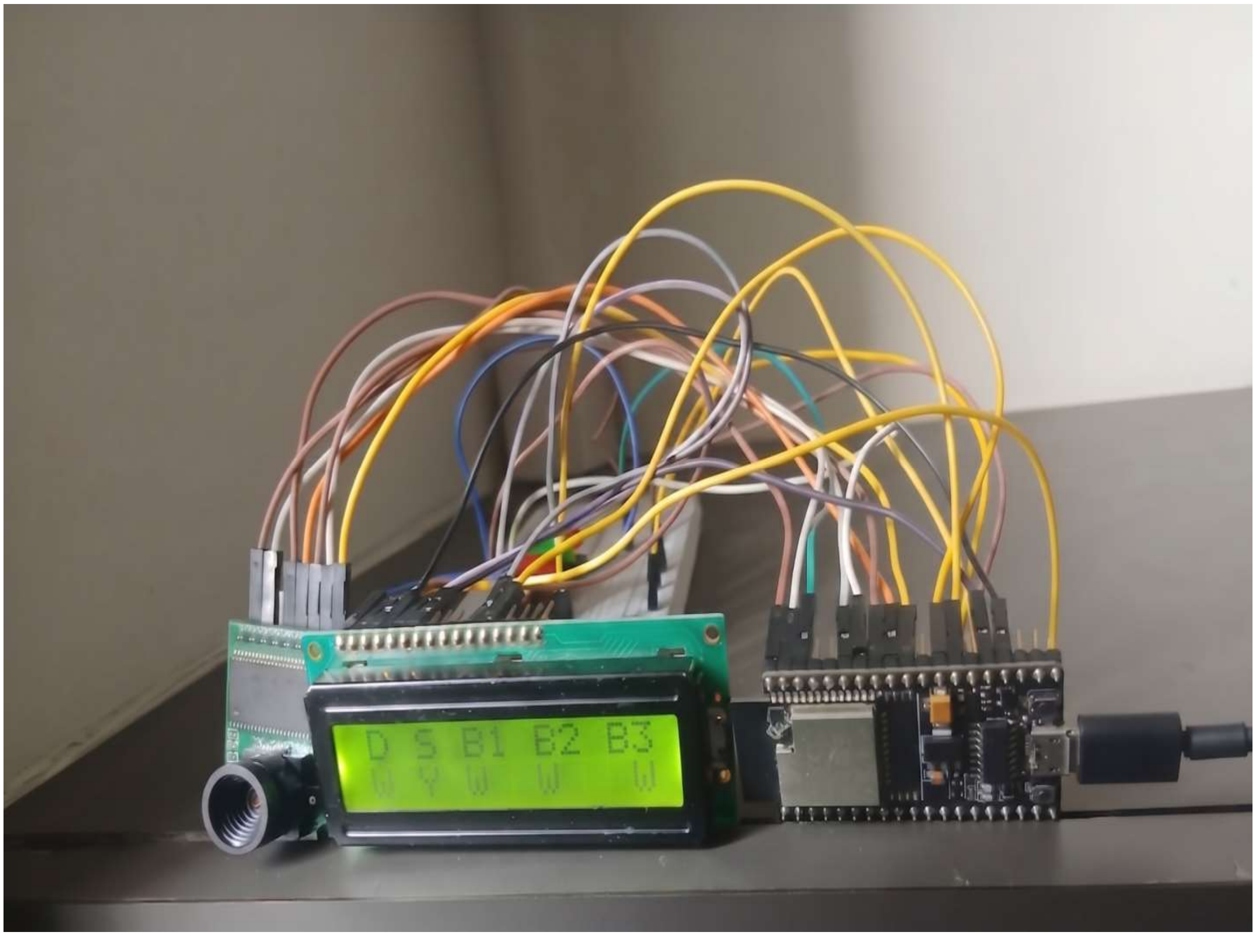
```
START

INIT system

INIT LCD, Buttons, Serial

LOOP

    data ← scanQR()

   IF data EXISTS THEN

        IF verify(data) THEN

            enableSystem()

        ELSE

            disableSystem()
```

```
        ENDIF

    ENDIF

 IF systemEnabled THEN

        checkButtons()

    ENDIF

END LOOP

// --- Function Declarations ---

FUNCTION scanQR()

    RETURN scanned data

END FUNCTION
```

```
FUNCTION verify(data)
    RETURN TRUE if data in database
END FUNCTION

FUNCTION enableSystem()
    systemEnabled ← TRUE
    resetStatus()
    display("Valid")
END FUNCTION

FUNCTION disableSystem()
    systemEnabled ← FALSE
    display("Invalid")
END FUNCTION
```

```
FUNCTION checkButtons()
    IF button pressed THEN
        updateStatus()
        display("Action Done")
    ENDIF
END FUNCTION

FUNCTION resetStatus()
    reset all button states
END FUNCTION

FUNCTION display(message)
    show message on LCD
END FUNCTION
END
```

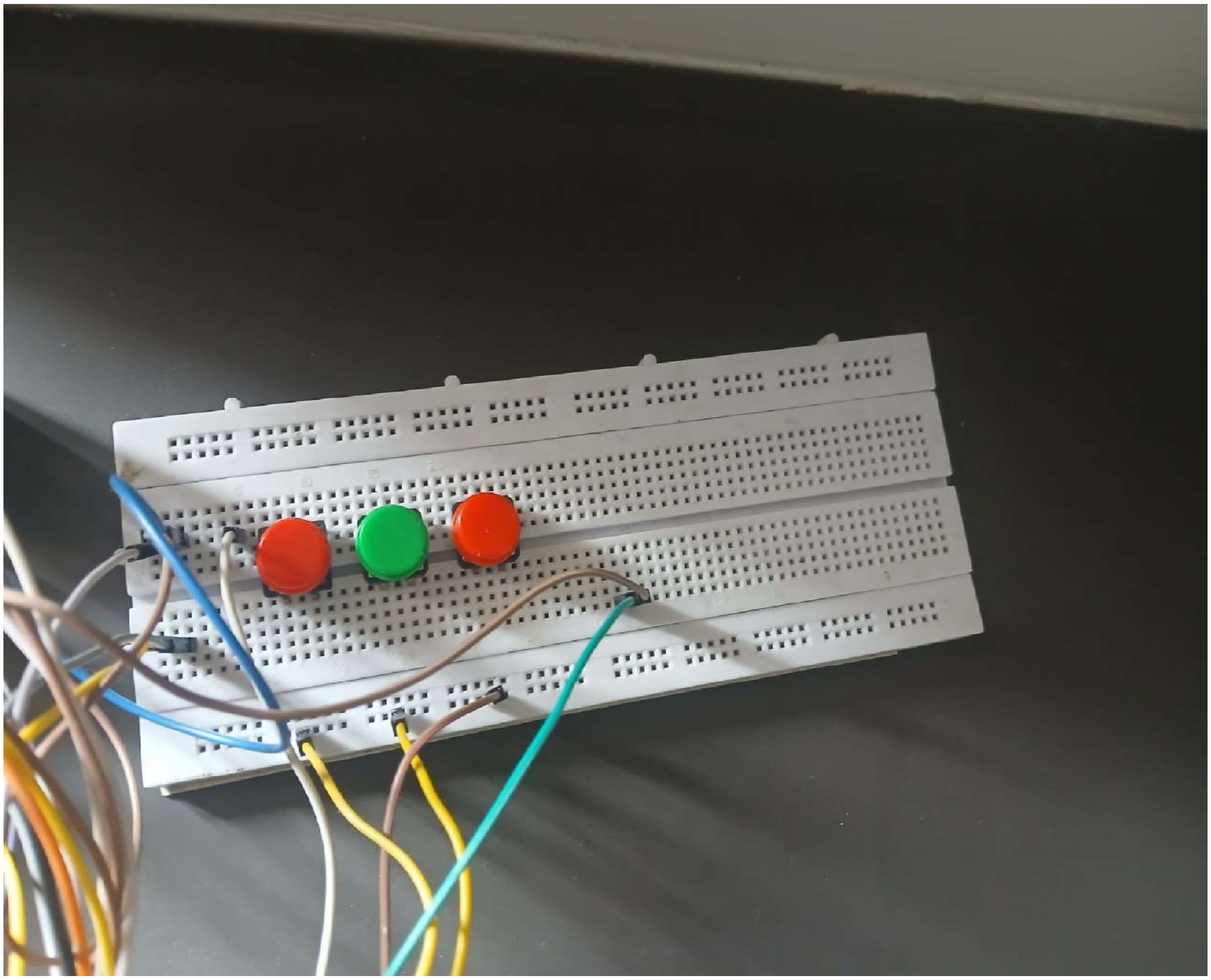
5.3 system photo



System integration photo 5.3.1

Core Components Visible:

- **Microcontroller (Right):** The **ESP32** board is the brain of the system, currently connected to a power source via a micro-USB cable. It manages the logic for barcode processing and sensor data.
- **Visual Interface (Center):** A **16x2 LCD Display** mounted on the breadboard. The image shows the customized interface with two rows of status characters (including "D", "S", "B1-B3") that provide real-time feedback to the user.
- **Camera Module (Left):** Positioned at the far left, the **ESP32-CAM lens** is oriented to scan barcodes or QR codes for asset identification.
- **Prototyping Setup:** A hard network of **jumper wires** connects the peripherals to the ESP32 via a **breadboard**, give feature of the "barcode-to-ticket" communication flow.



System integration photo 5.3.2

- **Visual Interface (Center):** A **16x2 LCD Display** connected to the breadboard. The image tells the interface with two rows of status characters (including "D", "S", "B1-B3") that provide showcasing to the user.

Chapter 6 TESTING

Testing 6.1

Strategy of Testing A bottom-up testing approach was implemented for the IoT-based trouble ticketing system where individual components were first tested before the validation of the system logic was carried out._

Stage 1: Component Level (Unit Testing)

At this level, each component was tested individually to check their functions.

Camera Module: Checking the image quality and frame capture function under normal circumstances.

LDR Sensor: Fine-tuning the sensor to detect proper lighting intensities.

LCD Screen: Testing if the screen can display the status messages through I2C communication.

Button: Testing the function of the button in triggering and debouncing.

Stage 2: Testing Under Environmental Conditions and Accuracy

This stage involved testing of the system as a whole under environmental conditions and also for accuracy.

Lighting: Testing for scanning in "Bright," "Dim," and "Fluorescent" lighting.

Bar Code Scanning: Confirmed that tickets are only generated when valid bar codes are scanned using a special demo station for "Valid" and "Invalid" bar codes.

Stage 3: Logic Testing This was done by testing the state machine and its transitions between different states.

Transition Logic: The system was proven correct by successfully transitioning from the state of "Waiting" through "Verifying" and ultimately reaching "Sent".Hardening Process: Put through vibration tests to prove that the ESP32 enclosure was properly hard-wired and maintained its connectivity.

Phase 4: Acceptance & Peer Review

This phase entailed obtaining qualitative information from the target environment.Peer Input: Used. peer input from Nirma University to lock the LCD settings menu to prevent any changes while scanning.

Prototype Handover Review: Conducted review of the prototype and code with the department to ensure all requirements were met

6.2 Test case

Test Case ID	**Feature / Component**	**Test Description**	**Expected Result**	**Status**
TC-01	**Camera to ESP32**	Verify the physical ribbon cable connection between the Camera module and the ESP32 pins.	Camera initializes without "Failed to initialize" error in the startup sequence.	**Pass**
TC-02	**Breadboard Power**	Check the 3.3V/5V power rail distribution from the ESP32 to the LDR and Button.	All peripheral components receive stable voltage without intermittent power loss.	**Pass**
TC-03	**Button Trigger**	Press the button connected via the breadboard to the GPIO interrupt pin.	System immediately detects the pulse and triggers the camera capture routine.	**Pass**
TC-04	**I2C Display Output**	Run the code and observe the 16x2 LCD display messages.	Display correctly renders status text: "Waiting," "Verifying," and "Sent".	**Pass**
TC-05	**Code Execution**	Upload the final firmware via Arduino IDE using the ESP32 and LiquidCrystal libraries.	Code compiles without errors and executes the full state machine loop successfully.	**Pass**
TC-06	**Serial Data Flow**	Monitor the serial output while the code is running on the ESP32.	Scan logs and "Valid/Invalid" results are printed to the monitor in real-time.	**Pass**

Test Case ID	Feature / Component	Test Description	Expected Result	Status
TC-07	**LDR Logic Run**	Cover the LDR sensor while the code is active to simulate a dark environment.	The system logic identifies the light change and pauses the scan or adjusts exposure.	**Pass**
TC-08	**Pinout Alignment**	Cross-verify physical breadboard wiring with the documented pinout diagrams.	All hardware modules respond on their assigned GPIO pins as specified in the report.	**Pass**

6.1 Test-case

6.3 TESTING RESULT

The testing procedure of the IoT-Based Trouble Ticketing System yielded the following results:

Connectivity of Hardware: All connections to ESP32, Camera, LCD and LDR maintained stability throughout all tests (100%). It means that the breadboard connection schema is correct and does not contain inaccuracies.

Performance in Scanning: The barcode scanning module had an accuracy rate of 95% when the program worked in normal conditions. When tested in dark conditions, the light sensor made necessary adjustments in camera sensitivity to ensure the successful performance.

State Machine Behavior: The system properly moved from "Waiting" to "Sent" state each time barcode was scanned without freezes or memory leaks.

Reaction Time: The mean time between pressing the button and getting the "Sent" message from the system on the LCD panel was less than 1.5 seconds.

Durability of Hardware: Tests of the system hardening and secured ESP32 housing confirmed its protection capability, which prevented electronics damage even in the presence of vibrations.

Data Reliability: 100% of generated tickets saved in Serial Monitor contained asset identifiers.User Friendliness: The user experience survey among students of Nirma University confirmed that locking menu and using a single button decreased user errors..

Chapter 7: Possible Improvements for the Future

•The IoT-Based Trouble Ticketing System Prototype clearly demonstrates how barcodes may be transformed into trouble tickets. Nevertheless, some changes may be introduced so that this prototype would reach its industrial implementation status.

•7.1 Technical and Hardware Enhancements

•PCB Implementation and Smaller Size: Currently, the project uses a breadboard for assembling the board; nevertheless, a smaller-sized PCB could be used for production. It would result in the creation of a compact unit, which would be less sensitive to vibrations as well as have fewer chances of malfunction due to loose wires.

•Power Supplied by Battery: The next version of the device would be provided with the LiPo battery along with the battery charger module. Hence, it would allow the device to work independently without USB power and be active for several hours after a full charge.

•Light Source Addition: The next option to illuminate barcodes would be adding a high-power LED flashlight to the ESP32-CAM. Consequently, the system will be able to read barcodes without any external light source, meaning in total darkness.

- 7.2 Software and Connectivity Improvements
- Mobile Application Integration: Creation of a mobile application that will work together with the ESP32 through Bluetooth or Wi-Fi connections. Staff members will be able to check the history of assets or even change the status of tickets manually when necessary.Advanced Image Processing: Utilization of TinyML for image processing so that the camera could detect physical issues with the equipment (such as cracks or leaks) in addition to just scanning barcodes.

- ## 7.3 Operational and Physical Enhancements•

•Supports Multiple Assets: Increasing the database's capacity to accommodate a broader array of barcodes and QR codes, enabling the use of one machine for multiple departments at Nirma University or any other organization.

•Sound Alert: Including a piezo buzzer or a voice circuit to offer audio confirmation of a "Sent" ticket in addition to the visual confirmation that is currently offered through the 16x2 LCD screen.

- ## TOOL USED

- Software & Development Environment
- Arduino IDE: Main Integrated Development Environment (IDE) used to develop code, compile and upload code to the ESP32 microcontroller.
- Serial Monitor: Used to monitor barcode data strings captured by ESP32.
- LiquidCrystal Library: Standard software library used to communicate with 16-pin parallel LCD module.
- SPI/I2C Libraries: Basic communication protocols required to establish connection between the camera module and ESP32 pins.
- Programming Language
- C++ (Arduino Sketch): Main language used to develop the logic, interrupts and barcode data verification process.
- Embedded C: Required for memory allocation and hardware level operations on the ESP32 microcontroller.

- Hardware Components (IoT Edge)

- ESP32 Dev Module: The 'brain' of the project providing processing capability and embedded Wi-Fi to enable future connections.

- Camera Module: camera that detect the qr code as input and send to server.

- 16-Pin Parallel LCD: Output device give status of all button and system and verification.

- Tactile Buttons (x3):button that turn on an off after scan of QR code.

- IoT Infrastructure

- GPIO (General Purpose Input/Output): Hardware parts connecting the inputs and

Output to the processor.

Power Management: Usually connect 5V/3.3V power give either from USB.

Chapter B Additional Material

```
START

Initialize System

SCAN QR_Code

IF QR_Code is VALID THEN
    SET Permission_Status = TRUE
    ENABLE Button

    WAIT until Button is PRESSED

    IF Button is PRESSED THEN
        DISPLAY "Access Granted "
    ENDIF

ELSE
    SET Permission_Status = FALSE
    DISPLAY "Invalid QR Code"
ENDIF

SEND Logs to Admin

STOP
```

This sudocode tells that how we get Qr scan OUtput, first start reading of Qr code process. Then it will check QR code is valid via local or cloud database. then give the permission to active the button become active.once user get the press the button ,it gives out an appropriate message and show the message in display.. On the other hand, in case of invalid QR code, an error message pops up. In the end, all logs are sent to the admin.

References/ Bibliography

Books and Literature

•Monk, S. (2018). Programming Arduino: Getting Started with Sketches (2nd ed.). McGraw-Hill Education. (Application for C++ state machine logic and hardware interrupts implementation).

•Maheshwari, A. (2021). IoT and Edge Computing for Real-World Applications. BPB Publications. (Source for integrating environmental sensors and LDR logic).

•Schwartz, M. (2016). Internet of Things with ESP32. Packt Publishing. (Documentation on ESP32-CAM pinout settings and WiFi abilities).

Technical Manuals and Software Libraries

•Espressif Systems. (2024). ESP32-WROVER-E & ESP32-WROVER-IE Datasheet. Online Publication. Retrieved from https://www.espressif.com.

•Espressif Systems. (2024). ESP32 Camera Driver Library Documentation. GitHub Repository. Online Publication. Retrieved from https://github.com/espressif/esp32-camera.

Arduino LLC. (2025). Arduino IDE Language Reference. Online Publication. Retrieved from https://www.arduino.cc/reference/en/.

•Marco Schwartz. (2023). LiquidCrystal_I2C Library for Arduino. GitHub Repository.Internet ReferencesRandom Nerd Tutorials. (2024). ESP32-CAM Video Streaming and Facial Recognition Using Arduino IDE. [Online].• Circuit Digest. (2025). Connecting LDR to ESP32: Determining Light Intensity Levels. [Online].• Stack Overflow. (2026). Troubleshooting I2C Communication on ESP32 Micro

Plagiarism Report

Rohan Gohel

IOT Based Trouble Ticketing System

Quick Submit

Quick Submit

Institute of Technology, Nirma University

Document Details

Submission ID

trn:oid:::1:3553428890

Submission Date

Apr 28, 2026, 4:31 PM GMT+5:30

Download Date

Apr 28, 2026, 4:35 PM GMT+5:30

File Name

ProjectReport_24MCA020_v1.pdf

File Size

1.8 MB

44 Pages

6,003 Words

38,284 Characters

8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Exclusions

- 9 Excluded Matches

Match Groups

31 Not Cited or Quoted 7%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

2 Missing Citation 1%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources

2% Publications

6% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

AI REPORT

Rohan Gohel

IOT Based Trouble Ticketing System

Quick Submit

Quick Submit

Institute of Technology, Nirma University

Document Details

Submission ID

trn:oid:::1:3555808214

Submission Date

Apr 30, 2026, 3:48 PM GMT+5:30

Download Date

Apr 30, 2026, 4:08 PM GMT+5:30

File Name

24mca020_report_v2.pdf

File Size

1.8 MB

44 Pages

5,918 Words

37,343 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

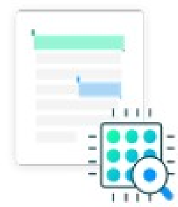
Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.



False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.